

Learning-Based Self-Healing Agents for Fault Detection and Recovery in Microservice Systems

Student 1: Eamon Cullimore - A21360701

Student 2: Hadrian Ging Hin Chong - A00053080

User Manual

Date Completed: 14/07/2026

1. Prerequisites

Before you begin, ensure your local machine or server meets the necessary requirements to handle a large microservice environment.

- **Docker:** Installed and running natively or via Docker Desktop.
- **Docker Compose:** Installed on your system.
- **Python >=3.13.14:** Python 3.13 or above is needed to run this system.
- **Pip:** Installed on your system to grab all requirements.
- **Git:** Installed to clone the repository.
- **Hardware:** At least 8GB to 16GB of RAM is highly recommended, as the microservice consists of over 40 services.

2. Architecture Overview

Train Ticket simulates an end-to-end ticket-booking workflow using diverse technology stacks. The primary architecture utilizes the following components:

Component	Technology Stack
Java	Spring Boot, Spring Cloud
Node.js	Express
Python	Django

Go	Webgo
Databases	MongoDB, MySQL
Tracing	Skywalking and Nacos Servers

3. Installation and Deployment

Deploying the system via Docker Compose is the most straightforward method for a quick start.

3.1: Clone the Repository

Open your terminal and clone the official repository to your local machine.

Bash

```
git clone
git@gitlab.computing.dcu.ie:cullime2/2026-mcm-cullime2-hadrian2.git;
cd 2026-mcm-cullime2-hadrian2/
```

3.2: Install requirements

Before anything can be done, cd into the \src folder and run the following command to install all requirements:

Bash

```
pip install -r requirements.txt
```

3.3: Start the System

Follow the steps below to start the system

1. Make sure you have Docker running on your desktop. This project was run using DockerDesktop.
2. Cd into the src\train-ticket directory, this will take the longest to start so do this first
 - a. In the directory create an .env file with the following variables:

Bash

```
NAMESPACE=codewisdom
```

```
TAG=latest
IMG_REPO=train-ticket
IMG_TAG=latest
```

3. Run the build command on your terminal to build the images, this will take a while.

Bash

```
docker-compose build
```

4. Then run the below command to start the microservice:
 - a. **Note:** The initial startup will take several minutes as Docker pulls the necessary images for all the microservices and database instances.

Bash

```
docker-compose -f docker-compose.yml up -d
```

3.4 Start Backend and Frontend for Tracing UI

To start the Tracing system follow the below steps

1. Open 2 more terminals and cd into the \self_healing_agent\src\UI\frontend folder in one terminal and \self_healing_agent\src in the other terminal.
2. From the self healing agent src directory run the following command

Bash

```
python -m uvicorn UI.backend.main:app --port 8000
```

- a. This starts the backend logic for the tracing.
3. From the UI\frontend directory run this command

Bash

```
python -m http.server 5500
```

- a. This starts up the Trace Dashboard UI.

4. Accessing the System Endpoints

Once all containers are successfully initialized and healthy, you can access the interfaces via your web browser.

Interface	URL	Purpose

Web UI	http://localhost:8080	The main user dashboard for booking train tickets.
Trace Dashboard	http://localhost:5500/trace_dashboard.html	The main UI to view microservice health and actions.
Sentinel Dashboard	http://localhost:5500/sentinel_dashboard.html	Secondary UI to see metrics and data.

5. Basic System Usage

UI Steps

To simulate a real ticketing workflow and generate telemetry data, follow these steps within the Web UI:

1. Navigate to http://localhost:8080 in your browser.
2. Login with the pre-populated credentials supplied by the repo owner
3. Head to the orders tab on the left and view the orders
 - a. **Note:** Due to hardware restrictions the booking system does not work as it requires too much memory and frequently breaks / crashes.
4. There you can pay an order.
5. Head to the execute flow tab and from there you can simulate collecting your ticket and entering the station (Using the ticket).

Simulating Faults

To simulate the fault injection process you first need to warm up the trace detection. To do this, run the following command in the \self_healing_agent\src directory:

Bash

```
python run_auto_query.py
```

That will warm up the traces on the dashboard.

Note: If it does not then run some of the services in the microservice UI a few times, you can spam the login button a few times to warm up the traces.

To inject faults you simply run the following command:

Bash

```
python main.py
```

This file will take a random failure and inject it into the service while also running a full runthrough of the system. You can then keep an eye out for any failures appearing on the Trace Dashboard.

6. System Shutdown

When you are finished benchmarking or testing, shut down the environment to free up your system's CPU and memory resources.

Bash

```
docker-compose down; docker volume prune -a -f
```

The prune command at the end deletes all stored volumes, this has caused issues in the past with data mismatches and the images searching for non-existent services.

7. Testing

To test this system navigate to the root \src directory (2026-mcm-cullime2-hadrian2/src) and run the following command to test all docker commands:

Bash

```
python -m pytest
```

8. Troubleshooting Common Issues

- **Containers crashing immediately:** This is almost always due to insufficient memory. Increase Docker's memory limit in your Docker Desktop settings to at least 8GB.
- **Port conflicts:** Ensure that ports 8080 (for the Web UI), 5500 (for Skywalking) and 8848, 9848 (for Nacos) are not currently being used by other local applications or services. Keep an eye on what the start command returns. The microservice will require a lot of ports.
- **Network timeouts during pull:** If the Docker image pulls fail halfway through, verify your internet connection and try again. Sometimes the pull will timeout, you can retry and it will work.
- **Missing Traces:** It can occasionally take a minute for the UI Dashboard to fully register. The best approach is to wait till you can login on the microservice UI before checking or expecting traces.

- **Issues with transformers and torch libraries:** It is very possible that torch and transformers will not work if you want to run files that import them, to do this you need Python 3.13 as mentioned in prerequisites. The command for this is python3.13 plus the filename to be executed.